| Estructura | Operaciones / Métodos | Tiempo (Time) | Espacio (Space) | Justificación Técnica |
|---|---|---|---|---|
| **Cola usando Pilas**(MyQueue) | push(x) | **O(1)** | **O(1)** | Inserta directamente en la pila de entrada (in_stack.append). |
| **Cola usando Pilas**(MyQueue) | pop(), peek() | **O(1) amortizado** | **O(1)** | Pasa elementos a out_stack solo cuando está vacía. Cada elemento se mueve máximo 1 vez. |
| **Cola usando Pilas**(MyQueue) | empty() | **O(1)** | **O(1)** | Verifica únicamente si ambas listas están vacías. |
| **Cola Circular**(MyCircularQueue) | enQueue(value) | **O(1)** | **O(1)** | Inserta en la posición dada por el índice tail usando el módulo (%). |
| **Cola Circular**(MyCircularQueue) | deQueue() | **O(1)** | **O(1)** | Avanza el índice head usando módulo (%) sin mover arreglos. |
| **Cola Circular**(MyCircularQueue) | Front(), Rear() | **O(1)** | **O(1)** | Acceso directo por índice a los punteros head o tail. |
| **Cola Circular**(MyCircularQueue) | isEmpty(), isFull() | **O(1)** | **O(1)** | Compara la variable interna de tamaño (size) contra la capacidad k. |

Reflexion:

La diferencia de rendimiento entre list.pop(0) y deque.popleft() se debe a su almacenamiento en memoria. Las listas usan espacios contiguos, por lo que al eliminar el primer elemento, todos los demás deben desplazarse físicamente, costando O(n). En cambio, el deque usa bloques enlazados, permitiendo eliminar el inicio actualizando solo los apuntadores en un tiempo constante de O(1) sin mover datos.

Como equipo, el problema que más nos costó trabajo fue diseñar la cola usando pilas (MyQueue) y la cola circular (MyCircularQueue). Discutir alternativas nos ayudó a entender cómo simular el comportamiento FIFO usando pilas LIFO y cómo aplicar el operador módulo para que los apuntadores dieran la vuelta en el carrusel sin desplazar arreglos, logrando operaciones eficientes.

Finalmente, la ventaja de re.fullmatch frente a re.match es que valida **toda** la cadena de inicio a fin de forma estricta. Por ejemplo, al evaluar tres dígitos con el patrón para la cadena "123abc", re.match da un resultado exitoso porque solo revisa el inicio, mientras que re.fullmatch falla correctamente al detectar que sobran los caracteres finales.